GEC Academy
Online Research Seminar

Course Title: Introduction to Ethereum and Cloud Blockchain Solutions Engineering

Instructor: Jean-Claude Franchitti

Session: 4

Assignment / Lab #4

Name: Yujing, Hua

Date: November 2, 2026

Email Address: Hi_yuuj-ng@outlook.com

GitHub Repository: https://github.com/p8zvvckk5s-jpg/ethereum-course-assignments

Statement of Effort and AI Assistance

I disclose the use of ChatGPT/Codex to interpret the assignment, research official technical documentation, draft explanations, develop and revise the accompanying code, and assist with automated local testing and document preparation. I reviewed the submitted material and take responsibility for its accuracy and for following the course policy on permitted assistance.

Signature: Hua Yujing

Assignment / Lab #4 Answers

Verification summary: all Solidity contracts compile with solc 0.5.1 and optimizer settings enabled. The XYZCoin test suite completed successfully with 11 passing tests on an isolated local Ganache chain. The Infura scanner is complete but intentionally contains no private API key; the final live run must be performed with the student's own Infura endpoint.

1. Blockchain Oracles

1(a) Definition and purpose

A blockchain oracle is a mechanism that supplies a smart contract with information that is not natively available in the blockchain's deterministic state. Examples include market prices, weather observations, sports results, shipping events, and authenticated enterprise data. Smart contracts cannot directly make ordinary web requests because every validating node must reproduce the same execution result. An oracle therefore bridges off-chain data and on-chain logic, normally by observing a request, obtaining and validating external data, and submitting a signed on-chain response.

1(b) Synchronous and asynchronous designs

A synchronous-style oracle exposes data that is already available on-chain when the consumer transaction runs. The contract reads a stored value and completes in one transaction. This is appropriate for values that are refreshed independently, such as a price feed. It is not truly synchronous with the external website; rather, the on-chain read is synchronous because the latest published value already exists.

An asynchronous oracle uses a request-and-callback workflow. Transaction 1 records a request and emits an event. An off-chain oracle service observes the event, obtains the external result, and later sends Transaction 2 to a callback function. The consumer must store request identifiers and continue only after the authenticated response arrives. This model is necessary when the answer must be generated after the request or when obtaining it takes time.

1(c) Advantages and disadvantages

Design	Advantages	Disadvantages
Synchronous-style stored feed	Simple consumer code; one transaction; immediate deterministic read; easy composition with other contracts.	May be stale; publisher pays continuously even without demand; cannot answer a newly generated query in the same transaction.
Asynchronous request/callback	Fresh, request-specific answers; supports complex or slow data retrieval; cost can be charged only when requested.	At least two transactions; latency; more state and failure handling; callback authentication and replay protection are required.

1(d) Update frequency under a USD 10 daily budget

Assumptions recorded on November 2, 2026: Etherscan's current network snapshot showed an average Ethereum transaction fee of approximately USD 0.11, and Ethereum documentation states that blocks are added about every 12 seconds. The assignment asks for a simple estimate using an average transaction cost, so each oracle update is treated as one average transaction.

```
maximum updates/day = floor(USD 10 / USD 0.11) = 90 updates
blocks/day          = 86,400 seconds / 12 seconds per block = 7,200 blocks
```

```
update interval     = 7,200 blocks / 90 updates = 80 blocks
```

Therefore, the oracle can be updated approximately once every 80 blocks (about every 16 minutes) while remaining within USD 10 per day. This is a planning estimate: fees and the ETH/USD exchange rate change, and the real oracle transaction may consume more or less gas than the network-wide average. A production scheduler should recalculate the interval from current fee data and keep a safety margin.

2. FlawedVoting and Unsigned-Integer Underflow

2(a) Intended functionality

The contract gives the deployer ten voting units. Calling vote(candidate, amountOfVotes) is intended to confirm that the caller has enough remaining voting units, deduct the chosen amount, and add that amount to the selected candidate's total. Public mappings make each account's remaining units and each candidate's total observable.

2(b) The bug

The condition remainingVotes[msg.sender] - amountOfVotes >= 0 is invalid for an unsigned integer. A uint cannot represent a negative number. In Solidity 0.5.1 arithmetic does not automatically revert: if the caller has 10 votes but requests 11, 10 - 11 wraps around to 2^256 - 1. That enormous value is still greater than or equal to zero, so the require statement always succeeds. The later subtraction underflows again, giving the caller an enormous remaining balance and allowing effectively unlimited voting.

2(c) Preventive idiom

Use checks-effects-interactions together with checked arithmetic. For this specific defect, compare before subtracting—require(remainingVotes[msg.sender] >= amountOfVotes)—or use a SafeMath subtraction function that performs this comparison internally. Modern Solidity 0.8+ performs checked arithmetic by default, but the assignment requires Solidity 0.5.1, so an explicit check or SafeMath is necessary.

2(d) Corrected implementation

```
pragma solidity 0.5.1;

library SafeMath {
    function sub(uint256 a, uint256 b) internal pure returns (uint256) {
        require(b <= a, "SafeMath: subtraction overflow");
        return a - b;
    }
}

contract FlawedVotingFixed {
    using SafeMath for uint256;
    mapping(address => uint256) public remainingVotes;
    mapping(uint256 => uint256) public votes;

    constructor() public { remainingVotes[msg.sender] = 10; }

    function vote(uint256 candidate, uint256 amountOfVotes) public {
        remainingVotes[msg.sender] = remainingVotes[msg.sender].sub(amountOfVotes);
        votes[candidate] += amountOfVotes;
    }
}
```

SafeMath.sub checks b <= a before computing a - b. An excessive vote therefore reverts and leaves both mappings unchanged. The full source is supplied as contracts/FlawedVotingFixed.sol.

3. Web3.js, Infura, and the First Contract-Creation Transaction

3(a) Web3.js

Web3.js is a JavaScript/TypeScript library for interacting with Ethereum nodes through JSON-RPC. It can read blocks and transactions, call contract functions, encode ABI data, subscribe to events, estimate gas, sign transactions, and broadcast signed transactions. In this exercise it is used to request full block objects and inspect every transaction's recipient field.

3(b) Infura setup

1. Create an Infura account and an Ethereum project in the Infura dashboard.
2. Copy the project's Mainnet HTTPS endpoint.
3. Copy .env.example to .env and replace YOUR_INFURA_PROJECT_ID with the student's endpoint.
4. Do not commit .env; it is excluded by .gitignore.

This report does not include or invent an Infura credential. Account creation and credential acceptance must be completed by the student because they involve a private third-party account.

3(c) Node project and dependency installation

```
mkdir assignment_lab4
cd assignment_lab4
npm init -y
npm install web3 dotenv
```

The submitted package.json pins web3 4.16.0 and dotenv 16.4.7. dotenv loads the private endpoint from a local .env file rather than embedding it in source code.

3(d) Program and result

Ethereum JSON-RPC defines a transaction's to field as null for contract creation. The submitted script scans blocks from the beginning, requests full transaction objects, and stops at the first transaction whose to field is null. Historical explorer data identifies block 46,402 as the first smart-contract deployment. A final Infura run will also print the creation transaction hash, creator address, and the created contract address obtained from the receipt.

```
const { Web3 } = require("web3");
const web3 = new Web3(process.env.INFURA_MAINNET_URL);

for (let first = startBlock; first <= endBlock; first += batchSize) {
  const numbers = /* the bounded block-number batch */;
  const blocks = await Promise.all(
    numbers.map((number) => web3.eth.getBlock(number, true))
  );
  for (const block of blocks) {
    const creation = block.transactions.find((tx) => tx.to === null);
    if (creation) {
      console.log(block.number, creation.hash);
      return creation.hash;
    }
  }
}
```

The complete executable implementation is scripts/find-first-contract.js. Run it with npm run find:first-contract after providing the private Infura endpoint.

3(e) Speed improvements

- Request hydrated blocks with getBlock(number, true), avoiding a separate RPC request for every transaction.
- Process a bounded batch concurrently with Promise.all instead of waiting for every block sequentially.
- Persist the most recently scanned block so an interrupted run can resume.
- Use an archive-capable endpoint; pruned nodes cannot return early Ethereum blocks.
- Respect provider rate limits with bounded concurrency and retry/backoff rather than unbounded requests.
- For repeated production queries, use an indexed blockchain dataset or explorer API rather than rescanning the chain.

4. Truffle ERC-20 Project: XYZCoin

4(a) Project initialization and folders

```
truffle init
```

Folder/file	Purpose
contracts/	Solidity source files, including interfaces and deployable contracts.
migrations/	Ordered JavaScript deployment scripts executed by Truffle.
test/	Mocha test files that use Truffle's contract abstraction and accounts.
build/contracts/	Generated ABI, bytecode, network metadata, and compiler output; not committed.
truffle-config.js	Network, compiler, optimizer, and test configuration.

4(b) Development network and compiler

```
development: { host: "127.0.0.1", port: 9545, network_id: "*" }
compilers: {
  solc: {
    version: "./node_modules/solc/soljson.js",
    settings: { optimizer: { enabled: true, runs: 200 } }
  }
}
```

The local compiler dependency is pinned to 0.5.1, satisfying the assignment and avoiding a silent compiler upgrade. The optimizer is enabled with 200 expected runs. A separate in-process test network is included so automated tests are reproducible without an external node.

4(c) ERC20 interface

```
interface ERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address owner) external view returns (uint256);
    function transfer(address to, uint256 value) external returns (bool);
    function transferFrom(address from, address to, uint256 value) external returns (bool);
    function approve(address spender, uint256 value) external returns (bool);
    function allowance(address owner, address spender) external view returns (uint256);
    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);
```

```
}
```

4(d) Token metadata, supply, and mappings

XYZCoin uses name “XYZ Coin”, symbol “XYZ”, zero decimals, and a fixed total supply of 1,000 whole tokens. The constructor assigns all 1,000 tokens to the deployer and emits the standard mint-style Transfer event from address(0). balances[address] stores each account's token amount. allowed[owner][spender] stores how many of an owner's tokens a spender is authorized to transfer.

```
string public constant name = "XYZ Coin";
string public constant symbol = "XYZ";
uint8 public constant decimals = 0;
uint256 private tokenTotalSupply = 1000;
mapping(address => uint256) public balances;
mapping(address => mapping(address => uint256)) public allowed;
```

4(e) Function implementation and events

transfer verifies the recipient and balance, updates both balances, and emits Transfer. approve writes the owner/spender allowance and emits Approval. transferFrom verifies the owner's balance and the caller's allowance, updates both balances and the remaining allowance, and emits Transfer. Zero-value transfers are valid and still emit Transfer as required by ERC-20. All functions return true on success.

```
function transferFrom(address from, address to, uint256 value)
    external returns (bool)
{
    require(to != address(0), "Recipient cannot be the zero address");
    require(balances[from] >= value, "Insufficient balance");
    require(allowed[from][msg.sender] >= value, "Insufficient allowance");
    balances[from] -= value;
    balances[to] += value;
    allowed[from][msg.sender] -= value;
    emit Transfer(from, to, value);
    return true;
}
```

The full implementation is supplied in contracts/XYZCoin.sol.

4(f) Migration

```
const XYZCoin = artifacts.require("XYZCoin");
module.exports = function (deployer) {
  deployer.deploy(XYZCoin);
};
```

4(g) Local deployment workflow

```
truffle develop
truffle(develop)> migrate --reset
```

truffle develop starts an isolated development blockchain with funded test accounts and opens the Truffle console. migrate --reset recompiles as needed and executes all migration scripts from the beginning. The result is a new Migrations contract and a new XYZCoin deployment whose 1,000-token supply belongs to the deployment account. These local addresses are disposable and are not public-network deployments.

4(h) Required Mocha tests

- Confirms name, symbol, decimals, and total supply.

- Confirms the creator initially owns all 1,000 tokens.
- Transfers tokens and checks both resulting balances.
- Sets an allowance and reads it back.
- Performs transferFrom and checks the recipient balance and reduced allowance.

4(i) truffle-assertions tests

- Insufficient-balance transfer reverts with the expected reason.
- Unauthorized transferFrom reverts for insufficient allowance.
- transfer emits Transfer for positive and zero values.
- transferFrom emits Transfer, including when the value is zero.
- approve emits Approval with the correct owner, spender, and value.

4(j) Verification result

```
Compiled successfully using:
  solc: 0.5.1+commit.c8a2cb62.Emscripten.clang

Contract: XYZCoin
  11 passing (1s)
```

The compilation and test run were completed locally on November 2, 2026. The µWS package printed a performance-only compatibility warning under the current Node runtime and automatically used its JavaScript fallback; compilation and all tests nevertheless completed successfully.

Submitted Programming Files

- contracts/ERC20.sol — ERC-20 interface
- contracts/XYZCoin.sol — XYZCoin implementation
- contracts/FlawedVotingFixed.sol — Corrected voting contract
- migrations/1_initial_migration.js — Truffle migration support
- migrations/2_xyz_coin.js — XYZCoin deployment
- test/xyz_coin.test.js — Mocha and truffle-assertions tests
- scripts/find-first-contract.js — Web3.js/Infura scanner
- truffle-config.js — Solidity 0.5.1, optimizer, and network settings
- .env.example — Safe endpoint template without credentials
- README.md — Installation and execution instructions

References

1. Ethereum Improvement Proposal 20, ERC-20 Token Standard: https://eips.ethereum.org/EIPS/eip-20
2. Ethereum.org, JSON-RPC API (transaction to is null for contract creation): https://ethereum.org/developers/docs/apis/json-rpc/
3. Ethereum.org, Block Explorers (approximately 12-second blocks): https://ethereum.org/developers/docs/data-and-analytics/block-explorers/
4. Web3.js Documentation: https://docs.web3js.org/
5. Truffle Configuration Reference (archived official documentation): https://archive.trufflesuite.com/docs/truffle/reference/configuration/
6. Etherscan Gas Tracker and network statistics: https://etherscan.io/gastracker/ and https://etherscan.io/charts
7. Ethereum Frontier historical account (first smart contract at block 46,402): https://hazelnutcloud.github.io/eth-book/part-1-genesis/chapter-05-frontier-launch.html
8. Assignment / Lab #4 specification supplied by the instructor.